Universidade de Aveiro

Departamento de Eletrónica, Telecomunicações e Informática

Project Title

Technical Report

Equipa

André Reis: 119814

Vasco Oliveira: 119113

Pedro Ferreira: 119240

Carlos Ramos: 119873

Dinis Malhado: 120420

Simão Pereira: 120459

Orientador: Diogo Gomes

March 19, 2026

Abstract

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context and Motivation

Anteriormente, a universidade disponibilizava a plataforma API (Academic Playground & Innovation), acessível no endereço api.web.ua.pt, que fornecia dados vitais para a comunidade, como a lotação de parques de estacionamento, senhas de serviços académicos, ementas das cantinas e informações sobre as unidades da universidade.

No entanto, este serviço teve de ser descontinuado devido a falhas críticas. Para resolver este problema, foi proposta a criação de uma nova API Gateway institucional, perfeitamente controlada, auditável e escalável.

1.2 Problem Statement

A descontinuação do serviço deve-se principalmente a vulnerabilidades de segurança e à falta de otimização do sistema. O sistema baseava-se numa versão de PHP muito antiga que já tinha atingido o seu "End of Life", o que significa que deixou de receber atualizações e ficou vulnerável a problemas de segurança.

Isto fez com que a integração entre sistemas fosse difícil e que a experiência da comunidade académica fosse prejudicada.

1.3 Objetivos

Os objetivos principais deste projeto são:

- Transcrever o código obsoleto e passar toda a infraestrutura para a linguagem Python, utilizando a tecnologia API Gateway WSO2 para ter a API funcional e segura;
- Garantir que todas as APIs já existentes correm de forma estável e funcional na

nova infraestrutura, de maneira a não prejudicar a experiência da comunidade académica;

- Desenvolver uma arquitetura escalável e fácil de manter, para que seja possível adicionar novas APIs que sejam relevantes no futuro.

1.4 Document Structure

This report is organised as follows. Chapter 2 presents related work, tools and technologies. Chapter 3 describes the requirements elicitation and system architecture. Chapter 4 outlines the expected results.

Chapter 2

Related Work, Tools and Technologies

2.1 Related Work

Two previous projects developed at the University of Aveiro are particularly relevant to this work: the *OPEN DATA UA* master thesis and the final report of the *Academic Playground & Innovation (API)* project. Together, they provide both the conceptual and practical foundations on which our project is built.

The *OPEN DATA UA* thesis surveys existing institutional information sources and web services at UA and proposes a service-oriented architecture based on REST and SOAP for exposing institutional data (e.g., cantina menus, academic information) in a standardized way. It also implements a web portal (APIn) that aggregates services from UA, PT Inovação and SAPO, and introduces an OAuth-based authorization server to protect access to private or sensitive web services. This work is mainly focused on identifying data sources, creating individual web services, and demonstrating how open data and secure access control can be applied within the university context.

The *Academic Playground & Innovation* final report documents the concrete implementation of that vision, namely the api.web.ua.pt portal and the supporting services running on services.web.ua.pt and identity.ua.pt. It describes in detail the API portal, the catalogue of services (UA, PT SAPO, PT Inovação), the documentation and wiki mechanisms, the statistics and administration interfaces, as well as the deployment of an OAuth 1.0a server integrated with the UA Identity Provider using SimpleSAMLphp.This report provides an operational view of how service publication, authentication, authorization and consumption were handled in practice in the first generation of the institutional API platform.

Our project can be seen as a next step in this evolution. While *OPEN DATA UA* and the API portal focused on exposing individual services and providing a web-based aggregation and documentation layer, they are tightly coupled to specific technologies and infrastructures that have since become obsolete. In contrast, our work aims at

designing a modern institutional API Gateway and governance model, aligned with current API management practices. Building on the lessons learned from these previous projects, we seek to provide a more sustainable, secure and governable integration layer for present and future UA systems.

2.1.1 Similar System / Approach A

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

2.1.2 Similar System / Approach B

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetuer at, consectetuer sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.

2.1.3 Summary and Comparison

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetuer a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetuer. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

2.2 Tools and Technologies

2.2.1 Technology / Framework A

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

2.2.2 Technology / Framework B

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

2.2.3 Technology / Framework C

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Chapter 3

Requirements Elicitation and Architecture

3.1 Requirements Elicitation

Tou a fazer isto, não mexam nisto pls :) by pedro

3.1.1 Functional Requirements

- **FR1 – Centralização e Exposição de Serviços:** Deve existir uma camada WSO2 que atue como API Gateway centralizada para expor os serviços da Universidade de Aveiro de forma uniforme.
- **FR2 – Identidade e Permissões:** O sistema tem de implementar mecanismos de autenticação e autorização, garantindo controlo de acesso por API, de acordo com o perfil de cada utilizador.
- **FR3 – Controlo e Monitorização:** A plataforma deve permitir o controlo e a monitorização detalhada de todos os acessos aos serviços, incluindo registo e rastreabilidade das chamadas efetuadas.
- **FR4 – Disponibilização de Documentação:** Deve estar disponível a documentação necessária para a utilização de cada API, incluindo descrição dos endpoints, parâmetros, autenticação e exemplos de utilização.
- **FR5 – Continuidade e Compatibilidade:** A plataforma deve manter os mesmos endpoints, assegurando compatibilidade com os sistemas já integrados e evitando interrupções de serviço.

3.1.2 Non-Functional Requirements

- **NFR1 – Segurança:** O sistema deve cumcomercialprir boas práticas de segurança, mitigando as vulnerabilidades identificadas na solução anterior e protegendo dados e acessos.
- **NFR2 – Desempenho:** Os pedidos à API devem ser processados, em condições normais de operação, em menos de 3 segundos.
- **NFR3 – Escalabilidade Técnica:** A API Gateway deve suportar aumento de carga e crescimento do número de consumidores sem degradação significativa do serviço.
- **NFR4 – Escalabilidade Funcional:** A introdução e integração de novas APIs deve ser simples, rápida e com impacto mínimo nos serviços existentes.
- **NFR5 – Manutenibilidade:** O sistema deve adotar boas práticas de desenvolvimento e manter o código legível, modular e bem documentado, de forma a facilitar a manutenção corretiva, evolutiva e preventiva.
- **NFR6 – Usabilidade da Interface:** O portal de documentação deve apresentar uma interface simples, clara e intuitiva, promovendo a adoção e utilização dos serviços disponibilizados.

3.2 System Architecture

The platform is organised into four distinct layers, as illustrated in Figure 3.1: the *User Interface*, the *WSO2 API Gateway*, the *FastAPI backend*, and the *Backend data sources*.

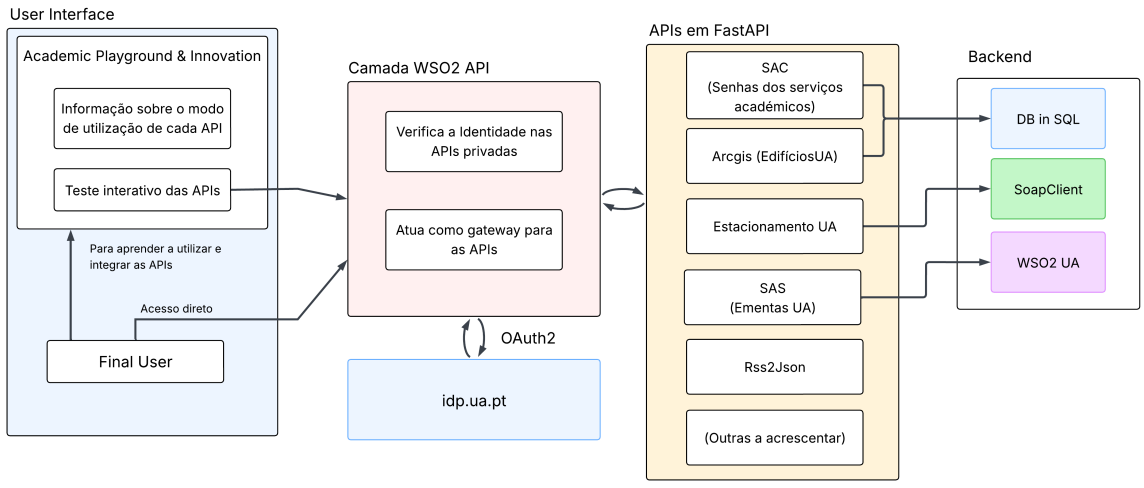

Figure 3.1: System Architecture Overview

3.2.1 User Interface — Academic Playground & Innovation

The user-facing layer is the *Academic Playground & Innovation* portal. It serves two purposes: providing documentation on how to use each API, and allowing interactive testing of all available endpoints directly in the browser. The goal here is to make it easy for people to learn how to integrate the APIs available in their own projetcs, as after they are familiar with the methods there is no real use for this layer.

The portal was developed using **Docusaurus**, a static site generator built on React and maintained by Meta. This choice is justified by its native support for Markdown and MDX, built-in documentation versioning, and ease of maintenance — updating content requires only editing text files, with no need to touch application code, this also facilitates escalating the number of APIs, as it's easy to make a complete documentation about all it's methods. The *Academic Playground & Innovation* portal provides interactive testing of all available endpoints directly in the browser, following the same approach used by platforms such as RapidAPI (wich heavily inspired this front-end). This is achieved by embedding **Stoplight Elements** into the Docusaurus site, which renders a full API explorer from the OpenAPI specification generated automatically by FastAPI.

Rather than communicating with the FastAPI backend directly, all test requests issued from the portal are routed through the **WSO2 API Gateway**, the same path taken by any external consumer. This is accomplished by setting the WSO2 public endpoint as the target server in the OpenAPI specification:

Listing 3.1: OpenAPI server configuration targeting WSO2

```
{
  "servers": [
    { "url": "https://api.ua.pt/v1" }
  ]
}
```

With this configuration, the portal's test interface reflects the real behaviour of the platform in production — including authentication, rate limiting, and request routing — rather than bypassing the gateway layer. For public APIs, such as the canteen menus, no credentials are required and requests can be issued immediately. For private APIs, Stoplight Elements supports OAuth2 flows natively, allowing users to authenticate with their institutional credentials via `idp.ua.pt` and have the resulting token attached automatically to all subsequent test requests.

3.2.2 WSO2 API Gateway Layer

All requests from the portal pass through a WSO2 API Manager instance, which acts as the central API Gateway for the platform. For the test requests explained in the previous layer to function correctly from a browser context, the WSO2 instance must be configured to include the appropriate `Access-Control-Allow-Origin` headers in its responses, permitting cross-origin requests originating from the portal's domain. This layer is responsible for:

- **Authentication and authorisation** — identity verification for private APIs is delegated to `idp.ua.pt` via OAuth2, this ensures institutional credentials are being requested correctly from UA's official identity provider;
- **Request routing** — the gateway forwards validated requests to the appropriate ServicesUAPT service;
- **Monitoring and rate limiting** — all inbound traffic is logged and can be controlled centrally without changes to the backend.

Public APIs (such as the canteen menus) are accessible without authentication, while APIs that expose sensitive data require a valid OAuth2 token issued by the university's identity provider.

3.2.3 FastAPI Backend - ServicesUAPT

The core logic of each service is implemented in Python using the FastAPI framework. This layer is completely replacing the old ServicesUAPT wich was very outdated. Each service is isolated in its own router module, making it straightforward to add, remove, or update individual APIs without affecting the rest of the platform. The services currently implemented or planned are:

- **SAC** — queue ticket numbers for the Academic Services office;
- **ArcGIS** — data about university buildings and campus units;
- **Estacionamento UA** — real-time parking availability;
- **SAS** — canteen menus, consumed from the UA WSO2 API and cached via Memcached;
- **Rss2Json** — generic RSS feed to JSON converter;
- (Additional services to be integrated in future iterations).

FastAPI was chosen because it generates OpenAPI-compliant documentation automatically, which integrates directly with the WSO2 gateway and removes the need to maintain separate API specifications.

3.2.4 Data Sources (Backend)

Depending on the service, the FastAPI layer consumes data from one of this sources:

- **CSV File** — used by the SAC service for persistent storage of queue ticket data, written and read directly from a local CSV file;
- **SOAP Client** — used by the SAC service to interface with the legacy university systems that expose SOAP/XML web services;
- **WSO2 UA** — the university's own API Gateway, consumed by services such as SAS to retrieve data like canteen menus and parking availability.

3.2.5 Deployment

The platform is containerised using Docker and deployed on the University of Aveiro's Kubernetes cluster, ensuring availability, scalability, and ease of rolling updates.

Chapter 4

Expected Results

Donec et nisl at wisi luctus bibendum. Nam interdum tellus ac libero. Sed sem justo, laoreet vitae, fringilla at, adipiscing ut, nibh. Maecenas non sem quis tortor eleifend fermentum. Etiam id tortor ac mauris porta vulputate. Integer porta neque vitae massa. Maecenas tempus libero a libero posuere dictum. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Aenean quis mauris sed elit commodo placerat. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Vivamus rhoncus tincidunt libero. Etiam elementum pretium justo. Vivamus est. Morbi a tellus eget pede tristique commodo. Nulla nisl. Vestibulum sed nisl eu sapien cursus rutrum.

4.1 Deliverables

- Deliverable 1 – Description;
- Deliverable 2 – Description;
- Deliverable 3 – Description.

4.2 Evaluation Criteria

Nulla non mauris vitae wisi posuere convallis. Sed eu nulla nec eros scelerisque pharetra. Nullam varius. Etiam dignissim elementum metus. Vestibulum faucibus, metus sit amet mattis rhoncus, sapien dui laoreet odio, nec ultricies nibh augue a enim. Fusce in ligula. Quisque at magna et nulla commodo consequat. Proin accumsan imperdiet sem. Nunc porta. Donec feugiat mi at justo. Phasellus facilisis ipsum quis ante. In ac elit eget ipsum pharetra faucibus. Maecenas viverra nulla in massa.

4.3 Project Schedule

Table 4.1: Planned project schedule

Phase	**Task**	**Start**	**End**
1	Requirements elicitation	Mar 2026	Mar 2026
2	Architecture design	Mar 2026	Apr 2026
3	Implementation	Apr 2026	Jun 2026
4	Testing and validation	Jun 2026	Jul 2026
5	Report writing and submission	Jul 2026	Jul 2026